X

x.com/affaanmustafa/article/2014040193557471352

January 21, 2026

The Longform Guide to Everything Claude Code

cogsec

@affaanmustafa

.

255K

In "The Shorthand Guide to Everything Claude Code", I covered the foundational setup: skills and commands, hooks, subagents, MCPs, plugins, and the configuration patterns that form the backbone of an effective Claude Code workflow. Its a setup guide and the base infrastructure.

cogsec

@affaanmustafa

.

Article
The Shorthand Guide to Everything Claude Code

Here's my complete setup after 10 months of daily use: skills, hooks, subagents, MCPs, plugins, and what actually works. Been an avid Claude Code user since the experimental rollout in Feb, and won...

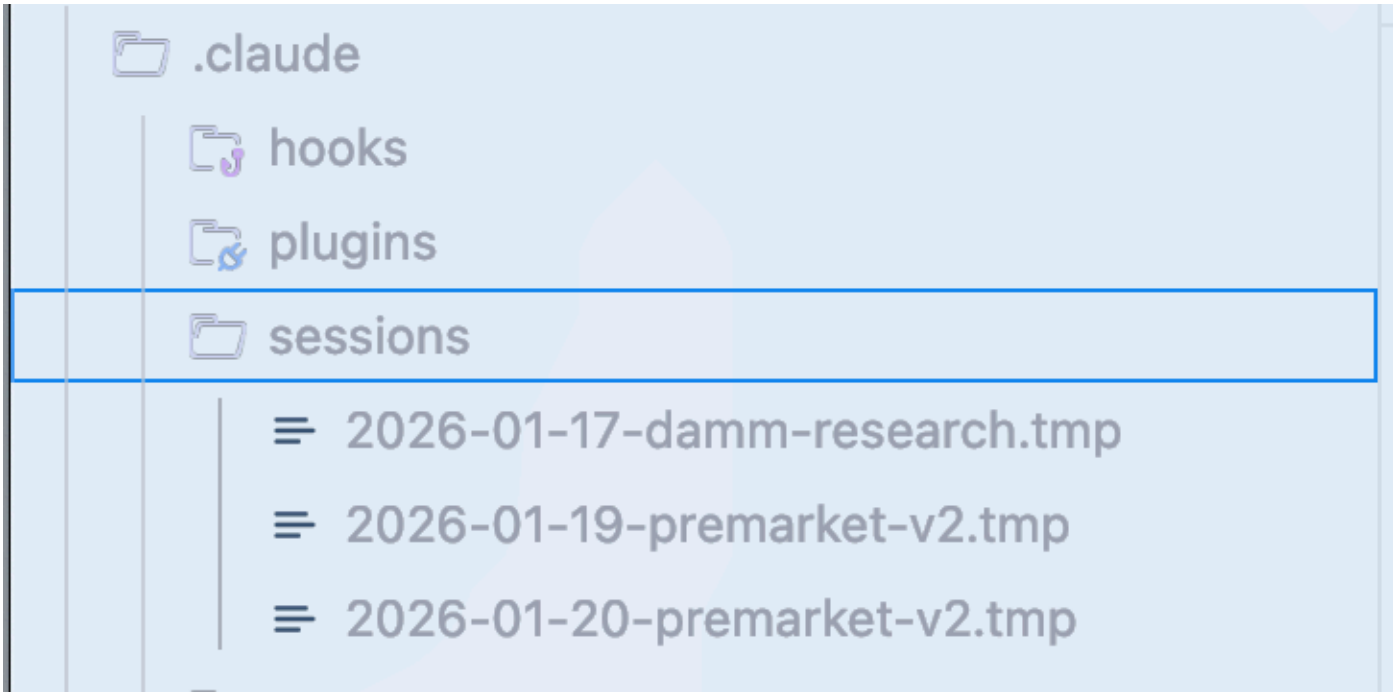
2.4M
This longform guide goes the techniques that separate productive sessions from wasteful ones. If you haven't read the
Shorthand Guide
, go back and set up your configs first. What follows assumes you have skills, agents, hooks, and MCPs already configured and working.
The themes here: token economics, memory persistence, verification patterns, parallelization strategies, and the compound effects of building reusable workflows. These are the patterns I've refined over 10+ months of daily use that make the difference between being plagued by context rot within the first hour, versus maintaining productive sessions for hours.

Everything covered in the shorthand and longform articles are available on github here:
everything-claude-code

Context & Memory Management

For sharing memory across sessions, a skill or command that summarizes and checks in on progress then saves to a `.tmp` file in your `.claude` folder and appends to it until the end of your session is the best bet. The next day it can use that as context and pick up where you left off,

create a new file for each session so you don't pollute old context into new work. Eventually you'll have a big folder of these session logs - just back it up somewhere meaningful or prune the session conversations you don't need.
Claude creates a file summarizing current state. Review it, ask for edits if needed, then start fresh. For the new conversation, just provide the file path. Particularly useful when you're hitting context limits and need to continue complex work. These files should contain - what approaches worked (verifiably with evidence), which approaches that were attempted did not work, which approaches have not been attempted and what's left to do.



Example of session storage ->
https://github.com/affaan-m/everything-claude-code/tree/main/examples/sessions
Clearing Context Strategically:

Once you have your plan set and context cleared (default option in plan mode in claude code now), you can work from the plan. This is useful when you've accumulated a lot of exploration context that's no longer relevant to execution. For strategic compacting, disable auto compact. Manually compact at logical intervals or create a skill that does so for you or suggests upon some defined criteria.

Strategic Compact Skill
(Direct Link):
(Embedded for quick reference)

bash

```
#!/bin/bash
# Strategic Compact Suggester
# Runs on PreToolUse to suggest manual compaction at logical intervals
#
# Why manual over auto-compact:
# - Auto-compact happens at arbitrary points, often mid-task
# - Strategic compacting preserves context through logical phases
# - Compact after exploration, before execution
# - Compact after completing a milestone, before starting next

COUNTER_FILE="/tmp/claude-tool-count-$$"
THRESHOLD=${COMPACT_THRESHOLD:-50}

# Initialize or increment counter
if [ -f "$COUNTER_FILE" ]; then
  count=$(cat "$COUNTER_FILE")
  count=$((count + 1))
  echo "$count" > "$COUNTER_FILE"
else
  echo "1" > "$COUNTER_FILE"
  count=1
fi

# Suggest compact after threshold tool calls
if [ "$count" -eq "$THRESHOLD" ]; then
  echo "[StrategicCompact] $THRESHOLD tool calls reached - consider /compact if
transitioning phases" >&2
fi
```

Hook it to PreToolUse on Edit/Write operations - it'll nudge you when you've accumulated enough context that compacting might help.

Advanced: Dynamic System Prompt Injection

One pattern I picked up and am trial running is: instead of solely putting everything in CLAUDE.md
(user scope) or `.claude/rules/` (project scope) which loads every session, use CLI flags to inject context dynamically.
bash

```
claude --system-prompt "$(cat memory.md)"
```

This lets you be more surgical about what context loads when. You can inject different context per session based on what you're working on.

Why this matters vs @ file references:

When you use `
@memory

.md` or put something in `.claude/rules/`, Claude reads it via the Read tool during the conversation - it comes in as tool output. When you use `--system-prompt`, the content gets injected into the actual system prompt before the conversation starts.
The difference is instruction hierarchy. System prompt content has higher authority than user messages, which have higher authority than tool results. For most day-to-day work this is marginal. But for things like strict behavioral rules, project-specific constraints, or context you absolutely need Claude to prioritize - system prompt injection ensures it's weighted appropriately.

Practical setup:

A valid way to do this is to utilize `.claude/rules/` for your baseline project rules, then have CLI aliases for scenario-specific context you can switch between:

bash

```
# Daily development
alias claude-dev='claude --system-prompt "$(cat ~/.claude/contexts/dev.md)"'

# PR review mode
alias claude-review='claude --system-prompt "$(cat ~/.claude/contexts/review.md)"'

# Research/exploration mode
alias claude-research='claude --system-prompt "$(cat ~/.claude/contexts/research.md)"'
```

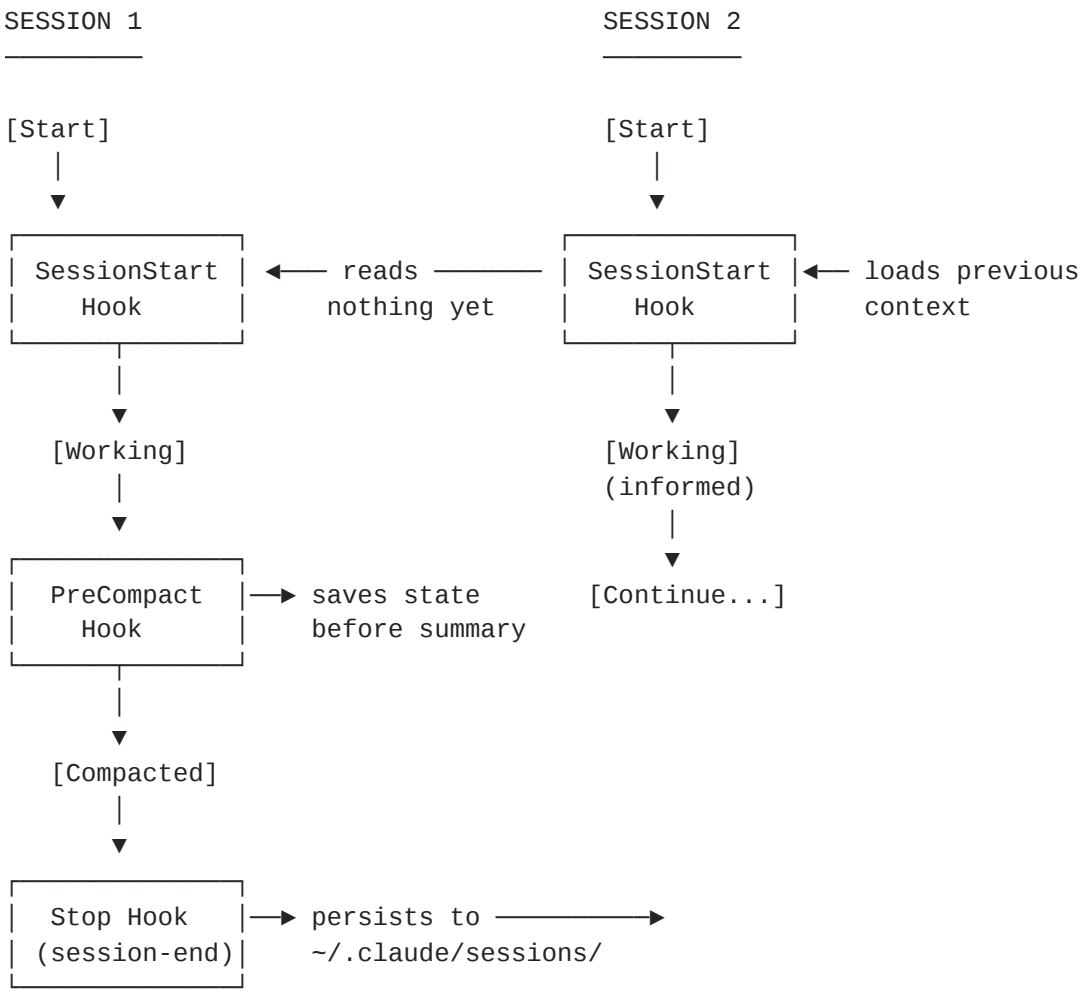
System Prompt Context Example Files
(Direct Link):

- dev.md
  focuses on implementation
- review.md
  on code quality/security
- research.md
  on exploration before acting

Again, for most things the difference between using `.claude/rules/context1.md` and directly appending `
context1.md
` to your system prompt is marginal. The CLI approach is faster (no tool call), more reliable (system-level authority), and slightly more token efficient. But it's a minor optimization and for many its more overhead than its worth.
Advanced: Memory Persistence Hooks

There are hooks most people don't know about or do but just don't really utilize that help with memory:

plaintext



- PreCompact Hook: Before context compaction happens, save important state to a file
- SessionComplete Hook: On session end, persist learnings to a file
- SessionStart Hook: On new session, load previous context automatically

[Memory Persistant Hooks]
(Direct Link):
(Embedded for quick reference)

json

```
{
  "hooks": {
    "PreCompact": [{
      "matcher": "*",
      "hooks": [{
        "type": "command",
        "command": "~/.claude/hooks/memory-persistence/pre-compact.sh"
      }]
    }],
    "SessionStart": [{
      "matcher": "*",
      "hooks": [{
        "type": "command",
        "command": "~/.claude/hooks/memory-persistence/session-start.sh"
      }]
    }],
    "Stop": [{
      "matcher": "*",
      "hooks": [{
        "type": "command",
        "command": "~/.claude/hooks/memory-persistence/session-end.sh"
      }]
    }]
  }
}
```

What these do:

- pre-compact.sh
  : Logs compaction events, updates active session file with compaction timestamp
- session-start.sh
  : Checks for recent session files (last 7 days), notifies of available context and learned skills
- session-end.sh
  : Creates/updates daily session file with template, tracks start/end times

Chain these together for continuous memory across sessions without manual intervention. This builds on the hook types from Article 1 (PreToolUse, PostToolUse, Stop) but targets the session lifecycle specifically.

Continuous Learning / Memory

We talked about continuous memory updating in the form of updating codemaps, but this applies to other things too such as learning from mistakes. If you've had to repeat a prompt multiple times and Claude ran into the same problem or gave you a response you've heard before this is applicable to you.

Most likely you needed to fire a second prompt to "resteer" and calibrate Claude's compass. This is applicable to any such scenario - those patterns must be appended to skills.

Now you can automatically do this by simply telling Claude to remember it or add it to your rules, or you can have a skill that does exactly that.

The Problem: Wasted tokens, wasted context, wasted time, your cortisol spikes as you frustratingly yell at claude to not do something that you already had told it not to do in a previous session.

The Solution: When Claude Code discovers something that isn't trivial- a debugging technique, a workaround, some project-specific pattern - it saves that knowledge as a new skill. Next time a similar problem comes up, the skill gets loaded automatically.

Continuous Learning Skill (Direct Link):
Why did I use a Stop hook instead of UserPromptSubmit? UserPromptSubmit runs on every single message you send - that's a lot of overhead, adds latency to every prompt, and frankly overkill for this purpose. Stop runs once at session end - lightweight, doesn't slow you down during the session, and evaluates the complete session rather than piecemeal.

Installation:

bash

```
# Clone to skills folder
git clone https://github.com/affaan-m/everything-claude-code.git
~/.claude/skills/everything-claude-code

# Or just grab the continuous-learning skill
mkdir -p ~/.claude/skills/continuous-learning
curl -sL https://raw.githubusercontent.com/affaan-m/everything-claude-
code/main/skills/continuous-learning/evaluate-session.sh > ~/.claude/skills/continuous-
learning/evaluate-session.sh
chmod +x ~/.claude/skills/continuous-learning/evaluate-session.sh
```

Hook Configuration
(Direct Link):
json

```
{
  "hooks": {
    "Stop": [
      {
        "matcher": "*",
        "hooks": [
          {
            "type": "command",
            "command": "~/.claude/skills/continuous-learning/evaluate-session.sh"
          }
        ]
      }
    ]
  }
}
```

This uses the Stop hook to run an activator script on every prompt, evaluating the session for knowledge worth extracting. The skill can also activate via semantic matching, but the hook ensures consistent evaluation.

The Stop hook triggers when your session ends - the script analyzes the session for patterns worth extracting (error resolutions, debugging techniques, workarounds, project-specific patterns etc.) and saves them as reusable skills in `~/.claude/skills/learned/`.

Manual Extraction with /learn:

You don't have to wait for session end. The repo also includes a `/learn` command you can run mid-session when you've just solved something non-trivial. It prompts you to extract the pattern right then, drafts a skill file, and asks for confirmation before saving. See
here
.
Session Log Pattern:

The skill expects session logs in `.tmp` files. The pattern is: `~/.claude/sessions/YYYY-MM-DD-topic.tmp` - one file per session with current state, completed items, blockers, key decisions, and context for next session. Example session files are in the repo at
examples/sessions/
.
Other Self-Improving Memory Patterns:

One approach from
@RLanceMartin
involves reflecting over session logs to distill user preferences - essentially building a "diary" of what works and what doesn't. After each session, a reflection agent extracts what went well, what failed, what corrections you made. These learnings update a memory file that loads in subsequent sessions.
Another approach from

@alexhillman
has the system proactively suggest improvements every 15 minutes rather than waiting for you to notice patterns. The agent reviews recent interactions, proposes memory updates, you approve or reject. Over time it learns from your approval patterns.

Token Optimization

I've gotten a lot of questions from price-elastic consumers, or those who run into limit issues frequently as power users. When it comes to token optimization there's a few tricks you can do.

Primary Strategy: Subagent Architecture

Primarily in optimizing the tools you use and subagent architecture designed to delegate the cheapest possible model that is sufficient for the task to reduce waste. You have a few options here - you could try trial and error and adapt as you go. Once you learn what is what, you can delegate to Haiku versus what you can delegate to Sonnet versus what you can delegate to Opus.

Benchmarking Approach (More Involved):

Another way that's a little more involved is that you can get Claude to set up a benchmark where you have a repo with well-defined goals and tasks and a well-defined plan. In each git worktree, have all subagents be of one model. Log as tasks are completed - ideally in your plan and in your tasks. You will have to use each subagent at least once.

Once you've completed a full pass and tasks have been checked off your Claude plan, stop and audit the progress. You can do this by comparing diffs, creating unit and integration and E2E tests that are uniform across all worktrees. That will give you a numerical benchmark based on cases passed versus cases failed. If everything passes on all, you'll need to add more test edge cases or increase the complexity of the tests. This may or may not be worth it, depending on how much this really even matters to you.

Model Selection Quick Reference:

Task Type	Model	Why
Exploration/search	Haiku	Fast, cheap, good enough for finding files
Simple edits	Haiku	Single-file changes, clear instructions
Multi-file implementation	Sonnet	Best balance for coding
Complex architecture	Opus	Deep reasoning needed
PR reviews	Sonnet	Understands context, catches nuance
Security analysis	Opus	Can't afford to miss vulnerabilities
Writing docs	Haiku	Structure is simple
Debugging complex bugs	Opus	Needs to hold entire system in mind

Hypothetical setup of subagents on various common tasks and reasoning behind the choices Default to Sonnet for 90% of coding tasks. Upgrade to Opus when first attempt failed, task spans 5+ files, architectural decisions, or security-critical code. Downgrade to Haiku when task is repetitive, instructions are very clear, or using as a "worker" in multi-agent setup. Frankly Sonnet 4.5 currently sits in a weird spot at $3 per million input tokens and $15 per million output tokens, the cost savings are ~ 66.7% over Opus, absolutely speaking thats a good saving but relatively its more or less insignificant to most people. Haiku and Opus combo makes the most sense as Haiku vs Opus is a 5x cost difference, compared to a 1.67x price difference against Sonnet.

Model	Base Input Tokens	5m Cache Writes	1h Cache Writes	Cache Hits & Refreshes	Output Tokens
Claude Opus 4.5	$5 / MTok	$6.25 / MTok	$10 / MTok	$0.50 / MTok	$25 / MTok
Claude Sonnet 4.5	$3 / MTok	$3.75 / MTok	$6 / MTok	$0.30 / MTok	$15 / MTok
Claude Haiku 4.5	$1 / MTok	$1.25 / MTok	$2 / MTok	$0.10 / MTok	$5 / MTok

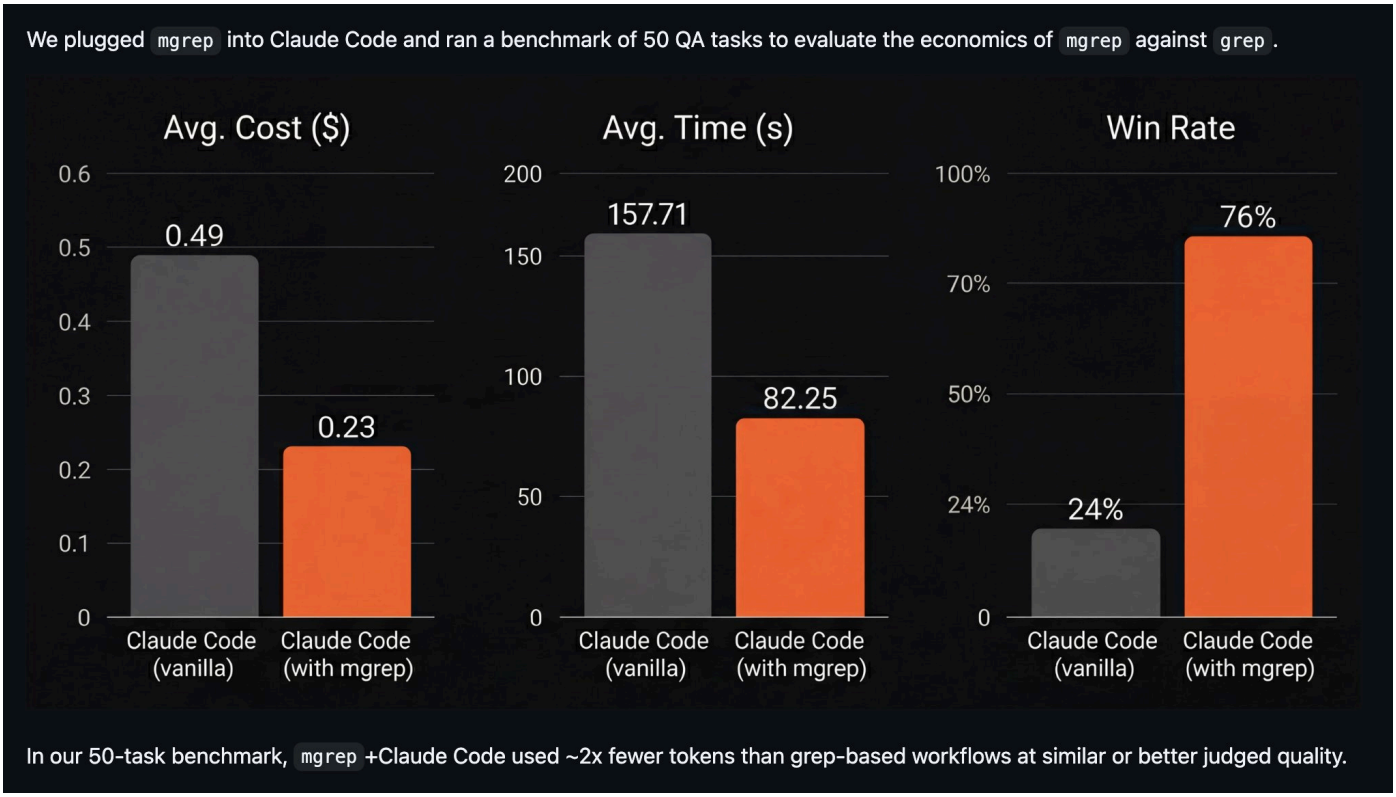
Source:
https://platform.claude.com/docs/en/about-claude/pricing
In your agent definitions, specify model:

yaml

```
---
name: quick-search
description: Fast file search
tools: Glob, Grep
model: haiku # Cheap and fast
---
```

Tool-Specific Optimizations:

Think about the tools that Claude calls the most frequently. For example, replace grep with mgrep - that on various tasks has an effective token reduction on average of around half compared to traditional grep or ripgrep, which is what Claude uses by default.



Source:
https://github.com/mixedbread-ai/mgrep/blob/main/README.md
Background Processes:

When applicable, run background processes outside Claude if you don't need Claude to process the entire output and be streaming live directly. This can be achieved easily with tmux (see
Shorthand Guide
and
Tmux Commands Reference (Direct Link)
. Take the terminal output and either summarize it or copy the part you need only. This will save on a lot of input tokens, which is where the majority of cost comes from - $5 per million tokens for Opus 4.5 and output is $25 per million tokens.
Modular Codebase Benefits:

Having a more modular codebase with reusable utilities, functions, hooks and more - with main files being in the hundreds of lines instead of thousands of lines - helps both in token optimization costs and getting a task done right on the first try, which correlate. If you have to prompt Claude multiple times you're burning through tokens, especially as it reads over and over on very long files. You'll notice it has to make a lot of tool calls to finish reading the file. Intermediary, it lets you know that the file is very long and it will continue reading. Somewhere along this process, Claude may lose some information. Also, stopping and rereading costs extra tokens. This can be avoided by having a more modular codebase. Example below ->

plaintext

```
root/
├── docs/                    # Global documentation
├── scripts/                 # CI/CD and build scripts
├── src/
│   ├── apps/                # Entry points (API, CLI, Workers)
│   │   ├── api-gateway/     # Routes requests to modules
│   │   └── cron-jobs/
│   │
│   ├── modules/             # The core of the system
│   │   ├── ordering/        # Self-contained "Ordering" module
│   │   │   ├── api/         # Public interface for other modules
│   │   │   ├── domain/      # Business logic & Entities (Pure)
│   │   │   ├── infrastructure/ # DB, External Clients, Repositories
│   │   │   ├── use-cases/   # Application logic (Orchestration)
│   │   │   └── tests/       # Unit and integration tests
│   │   │
│   │   ├── catalog/         # Self-contained "Catalog" module
│   │   │   ├── domain/
│   │   │   └── ...
│   │   │
│   │   └── identity/        # Self-contained "Auth/User" module
│   │       ├── domain/
│   │       └── ...
│   │
│   ├── shared/              # Code used by EVERY module
│   │   ├── kernel/          # Base classes (Entity, ValueObject)
│   │   ├── events/          # Global Event Bus definitions
│   │   └── utils/           # Deeply generic helpers
│   │
│   └── main.ts              # Application bootstrap
├── tests/                   # End-to-End (E2E) global tests
├── package.json
└── README.md
```

Lean Codebase = Cheaper Tokens:

This may be obvious, but the leaner your codebase is, the cheaper your token cost will be. It's crucial to identify dead code by using skills to continuously clean the codebase by refactoring using skills and commands. Also at certain points, I like to go through and skim the whole codebase looking for things that stand out to me or look repetitive, manually piece together that context, and then feed that into Claude alongside the refactor skill and dead code skill.

System Prompt Slimming (Advanced):

For the truly cost-conscious: Claude Code's system prompt takes ~18k tokens (~9% of 200k context). This can be reduced to ~10k tokens with patches, saving ~7,300 tokens (41% of static overhead). See YK's
system-prompt-patches
if you want to go this route, personally I don't do this.

Verification Loops and Evals

Evaluations and harness tuning - depending on the project, you'll want to use some form of observability and standardization.

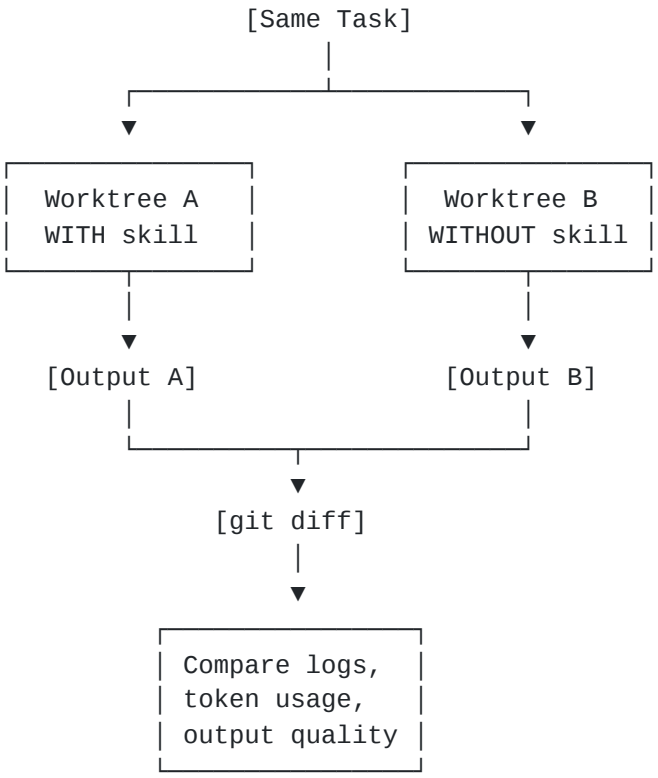
Observability Methods:

One way to do this is to have tmux processes hooked to tracing the thinking stream and output whenever a skill is triggered. Another way is to have a PostToolUse hook that logs what Claude specifically enacted and what the exact change and output was.

Benchmarking Workflow:

Compare that to asking for the same thing without the skill and checking the output difference to benchmark relative performance:

plaintext

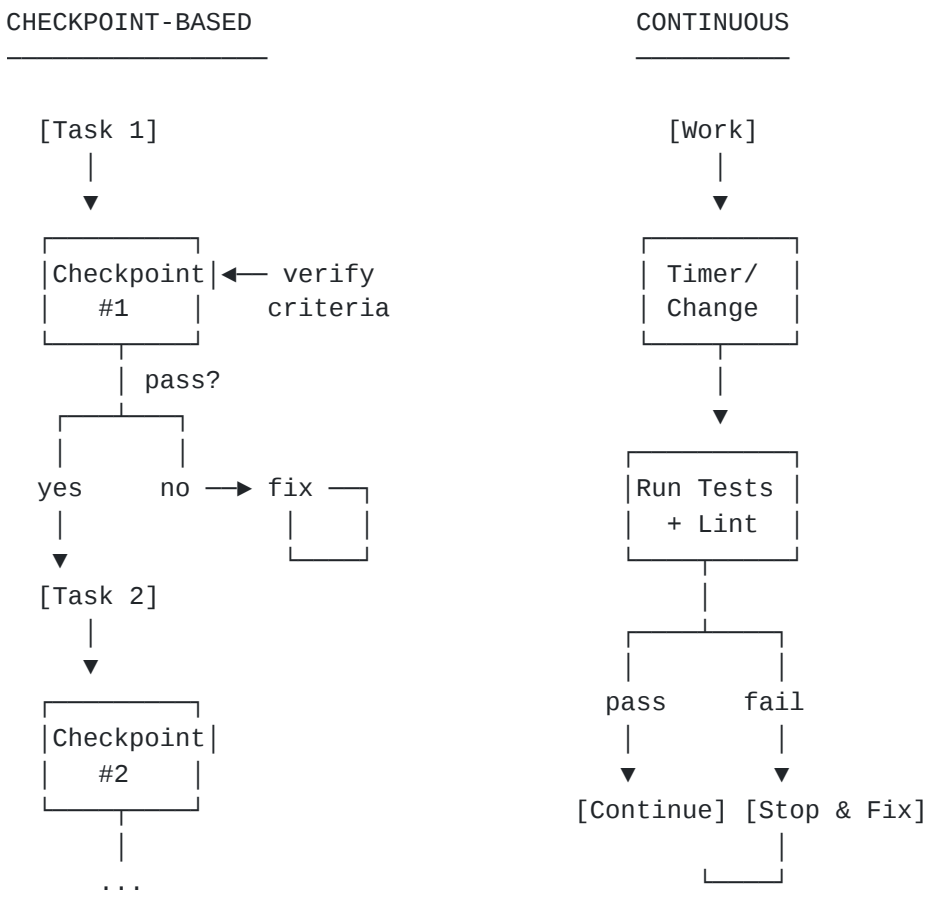


Fork the conversation, initiate a new worktree in one of them without the skill, pull up a diff at the end, see what was logged. This ties in with the Continuous Learning and Memory section.

Eval Pattern Types:

More advanced eval and loop protocols enter here. The split is between checkpoint-based evals and RL task-based continuous evals.

plaintext



Checkpoint-Based Evals:

- Set explicit checkpoints in your workflow
- Verify against defined criteria at each checkpoint
- If verification fails, Claude must fix before proceeding
- Good for linear workflows with clear milestones

Continuous Evals:

- Run every N minutes or after major changes
- Full test suite, build status, lint

- Report regressions immediately
- Stop and fix before continuing
- Good for long-running sessions

The deciding factor is the nature of your work. Checkpoint-based works for feature implementation with clear stages. Continuous works for exploratory refactoring or maintenance where you don't have clear milestones.

I would say with some intervention, the verification approach is enough to avoid most tech debt. Having Claude validate after it completes tasks by running the skills and PostToolUse hooks aids in that. Having the continuous codemap updating also helps because it keeps a log of changes and how the codemap evolves over time, serving as a source of truth outside just the repo itself. With strict rules, Claude will avoid creating random .md files cluttering everything as well as duplicate files for similar code and leaving a wasteland of dead code.

Grader Types (From Anthropic - Direct Link):
Code-Based Graders: String match, binary tests, static analysis, outcome verification. Fast, cheap, objective, but brittle to valid variations.

Model-Based Graders: Rubric scoring, natural language assertions, pairwise comparison. Flexible and handles nuance, but non-deterministic and more expensive.

Human Graders: SME review, crowdsourced judgment, spot-check sampling. Gold standard quality, but expensive and slow.

Key Metrics:

plaintext

```
pass@k: At least ONE of k attempts succeeds
        ┌─────────────────────────────────────┐
        │  k=1: 70%  k=3: 91%  k=5: 97%       │
        │  Higher k = higher odds of success  │
        └─────────────────────────────────────┘

pass^k: ALL k attempts must succeed
        ┌─────────────────────────────────────┐
        │  k=1: 70%  k=3: 34%  k=5: 17%       │
        │  Higher k = harder (consistency)    │
        └─────────────────────────────────────┘
```

Use pass@k when you just need it to work and any verifying feedback is enough. Use pass^k when consistency is essential and you need near deterministic output consistency (in terms of results/quality/style).

Building an Eval Roadmap (from the same Anthropic guide):

1. Start early - 20-50 simple tasks from real failures
2. Convert user-reported failures into test cases
3. Write unambiguous tasks - two experts should reach same verdict
4. Build balanced problem sets - test when behavior should AND shouldn't occur
5. Build robust harness - each trial starts from clean environment
6. Grade what agent produced, not the path it took
7. Read transcripts from many trials
8. Monitor for saturation - 100% pass rate means add more tests

---

Parallelization

When forking conversations in a multi-Claude terminal setup, make sure the scope is well-defined for the actions in the fork and the original conversation. Aim for minimal overlap when it comes to code changes. Choose tasks that are orthogonal to each other to prevent the possibility of interference.

My Preferred Pattern:

Personally, I prefer the main chat to be working on code changes and the forks I do are for questions I have about the codebase and its current state, or to do research on external services such as pulling in documentation, searching GitHub for an applicable open source repo that would help in the task, or other general research that would be helpful.

On Arbitrary Terminal Counts:

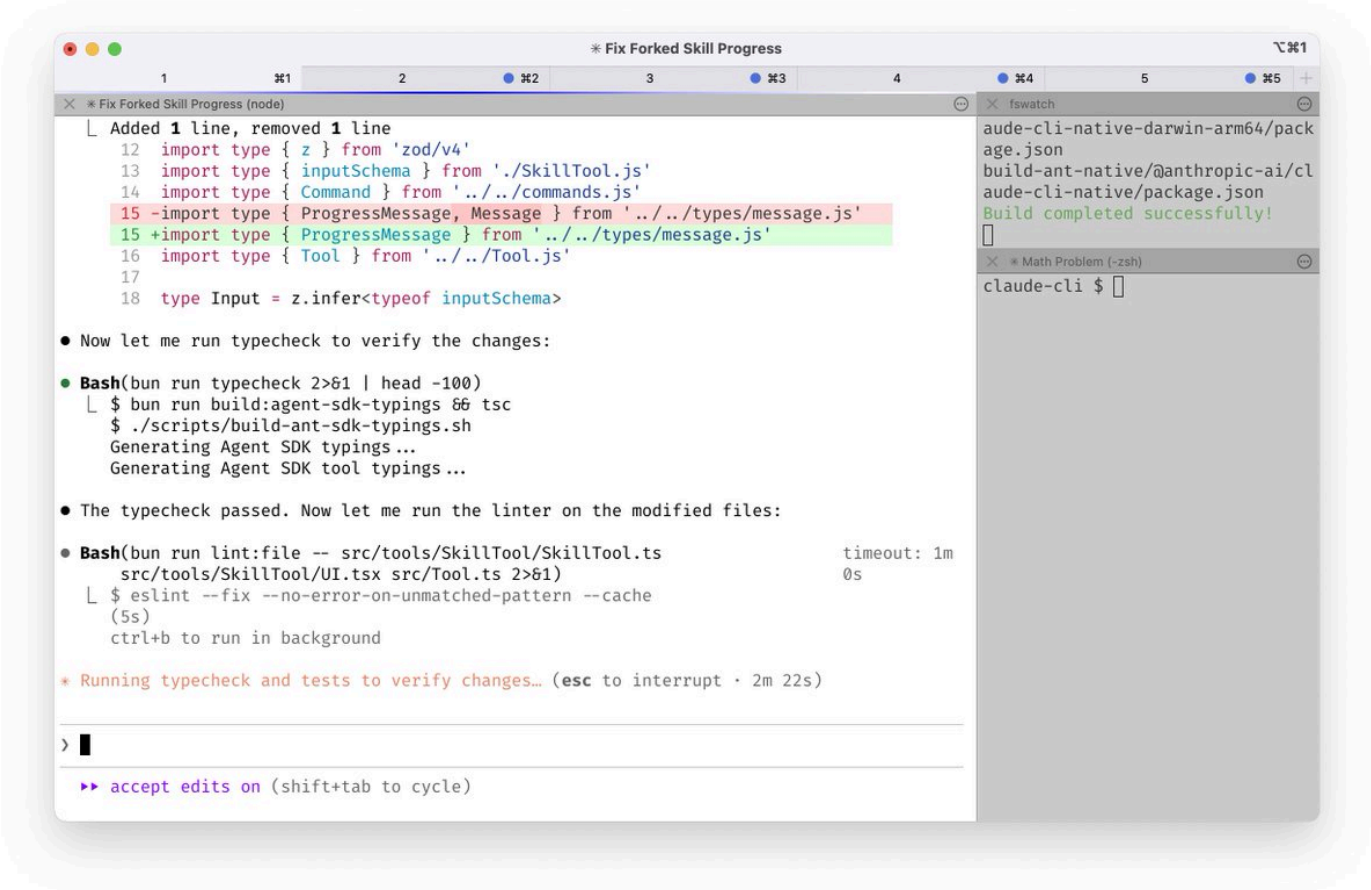
Boris
@bcherny
(the legend who created claude code) has some tips on parallelization that I agree and disagree with. He's suggested things like running 5 Claude instances locally and 5 upstream. I advise against setting arbitrary terminal amounts like this. The addition of a terminal and the addition of an instance should be out of true necessity and purpose. If you can take care of that task using a script, use a script. If you can stay in the main chat and get Claude to spin up an instance in tmux and stream it in a separate terminal that way, do that.

Boris Cherny

@bcherny

.

Replying to
@bcherny
1/ I run 5 Claudes in parallel in my terminal. I number my tabs 1-5, and use system notifications to know when a Claude needs input https://code.claude.com/docs/en/terminal-config#iterm-2-system-notifications…



1M

Your goal really should be: how much can you get done with the minimum viable amount of parallelization.

For most newcomers, I'd even stay away from parallelization until you get the hang of just running a single instance and managing everything within that. I'm not advocating to handicap yourself - I'm saying just be careful. Most of the time, even I only use 4 terminals or so total. I find I'm able to do most things with just 2 or 3 instances of Claude open usually.

When Scaling Instances:

IF you are to begin scaling your instances AND you have multiple instances of Claude working on code that overlaps with one another, it's imperative you use git worktrees and have a very well-defined plan for each. Furthermore, to not get confused or lost when resuming sessions as to which git worktree is for what (beyond the names of the trees), use `/rename <name here>` to name all your chats.

Git Worktrees for Parallel Instances:

bash

```
# Create worktrees for parallel work
git worktree add ../project-feature-a feature-a
git worktree add ../project-feature-b feature-b
git worktree add ../project-refactor refactor-branch

# Each worktree gets its own Claude instance
cd ../project-feature-a && claude
```

Benefits:

- No git conflicts between instances
- Each has clean working directory
- Easy to compare outputs
- Can benchmark same task across different approaches

The Cascade Method:

When running multiple Claude Code instances, organize with a "cascade" pattern:

- Open new tasks in new tabs to the right
- Sweep left to right, oldest to newest
- Maintain consistent direction flow
- Check on specific tasks as needed

- Focus on at most 3-4 tasks at a time - more than that and mental overhead increases faster than productivity

---

Groundwork

When starting fresh, the actual foundation matters a lot. This should be obvious but as complexity and size of codebase increases, tech debt also increases. Managing it is incredibly important and not as difficult if you follow a few rules. Besides setting up your Claude effectively for the project at hand (see the shorthand guide).

The Two-Instance Kickoff Pattern:

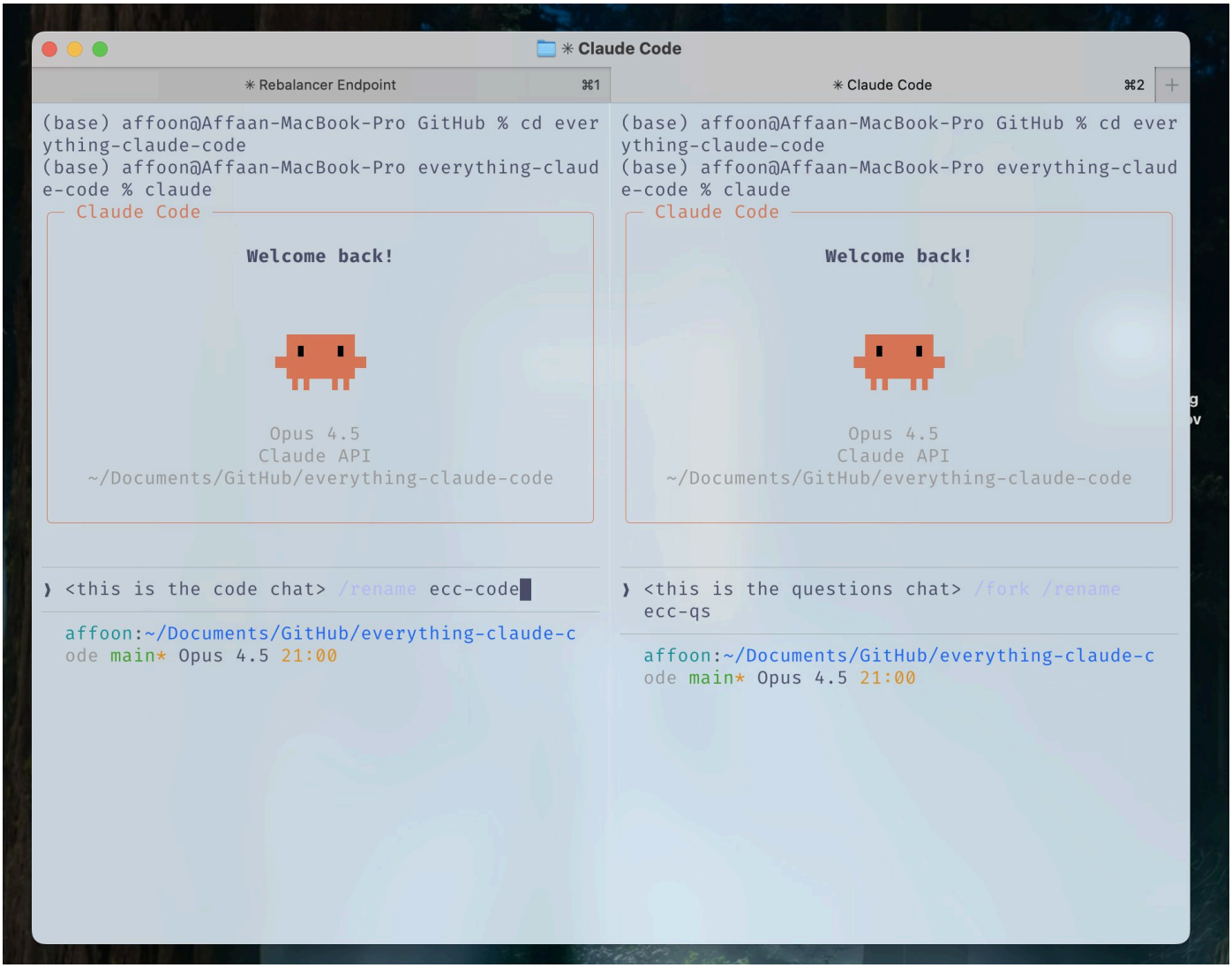
For my own workflow management (not necessary but helpful), I like to start an empty repo with 2 open Claude instances.

Instance 1: Scaffolding Agent

- Going to lay down the scaffold and groundwork
- Creates project structure
- Sets up configs (CLAUDE.md , rules, agents - everything from the shorthand guide)
- Establishes conventions
- Gets the skeleton in place

Instance 2: Deep Research Agent

- Connects to all your services, web search, etc.
- Creates the detailed PRD
- Creates architecture mermaid diagrams
- Compiles the references with actual clips from actual documentation



Starting Setup: Left Terminal for Coding, Right Terminal for Questions - use /rename and /fork.

What you need minimally to start is fine - it's quicker that way over Context7 every time or feeding in links for it to scrape or using Firecrawl MCP sites. All those work when you are already knee deep in something and Claude is clearly getting syntax wrong or using dated functions or endpoints.

llms.txt Pattern:

If available, you can find an llms.txt on many documentation references by doing `/llms.txt` on them once you reach their docs page. Here's an example:
https://www.helius.dev/docs/llms.txt
This gives you a clean, LLM-optimized version of the documentation that you can feed directly to Claude.

Philosophy: Build Reusable Patterns

One insight from

@omarsar0
that I fully endorse: "Early on, I spent time building reusable workflows/patterns. Tedious to build, but this had a wild compounding effect as models and agent harnesses improved."
What to invest in:

- Subagents (the shorthand guide)
- Skills (the shorthand guide)
- Commands (the shorthand guide)
- Planning patterns
- MCP tools (the shorthand guide)
- Context engineering patterns

Why it compounds (
@omarsar0
): "The best part is that all these workflows are transferable to other agents like Codex." Once built, they work across model upgrades. Investment in patterns > investment in specific model tricks.

Best Practices for Agents & Sub-Agents

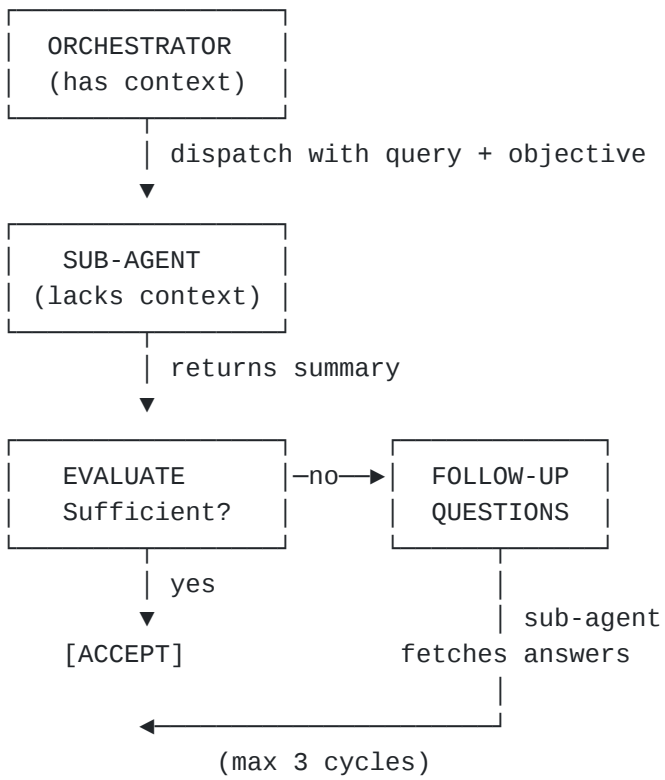
In the shorthand guide, I listed the subagent structure - planner, architect, tdd-guide, code-reviewer, etc. In this part we focus on the orchestration and execution layer.

The Sub-Agent Context Problem:

Sub-agents exist to save context by returning summaries instead of dumping everything. But the orchestrator has semantic context the sub-agent lacks. The sub-agent only knows the literal query, not the PURPOSE/REASONING behind the request. Summaries often miss key details.

The analogy from
@PerceptualPeak
: "Your boss sends you to a meeting and asks for a summary. You come back and give him the rundown. Nine times out of ten, he's going to have follow-up questions. Your summary won't include everything he needs because you don't have the implicit context he has."
Iterative Retrieval Pattern:

plaintext



To fix this, make the orchestrator:

- Evaluate every sub-agent return
- Ask follow-up questions before accepting it
- Sub-agent goes back to source, gets answers, returns
- Loop until sufficient (max 3 cycles to prevent infinite loops)

Pass objective context, not just the query. When dispatching a subagent, include both the specific query AND the broader objective. This helps the subagent prioritize what to include in its summary.

Pattern: Orchestrator with Sequential Phases

markdown

```
Phase 1: RESEARCH (use Explore agent)

- Gather context
- Identify patterns
- Output: research-summary.md

Phase 2: PLAN (use planner agent)

- Read research-summary.md
- Create implementation plan
- Output: plan.md

Phase 3: IMPLEMENT (use tdd-guide agent)

- Read plan.md
- Write tests first
- Implement code
- Output: code changes

Phase 4: REVIEW (use code-reviewer agent)

- Review all changes
- Output: review-comments.md

Phase 5: VERIFY (use build-error-resolver if needed)

- Run tests
- Fix issues
- Output: done or loop back
```

Key rules:

1. Each agent gets ONE clear input and produces ONE clear output
2. Outputs become inputs for next phase
3. Never skip phases - each adds value
4. Use `/clear` between agents to keep context fresh
5. Store intermediate outputs in files (not just memory)

Agent Abstraction Tierlist (from
@menhguin
):
Tier 1: Direct Buffs (Easy to Use)

- Subagents - Direct buff for preventing context rot and ad-hoc specialization. Half as useful as multi-agent but MUCH less complexity

- Metaprompting - "I take 3 minutes to prompt a 20-minute task." Direct buff - improves stability and sanity-checks assumptions
- Asking user more at the beginning - Generally a buff, though you have to answer questions in plan mode

Tier 2: High Skill Floor (Harder to Use Well)

- Long-running agents - Need to understand shape and tradeoff of 15 min task vs 1.5 hour vs 4 hour task. Takes some tweaking and is obviously very long trial-and-error
- Parallel multi-agent - Very high variance, only useful on highly complex OR well-segmented tasks. "If 2 tasks take 10 minutes and you spend an arbitrary amount of time prompting or god forbid, merge changes, it's counterproductive"
- Role-based multi-agent - "Models evolve too fast for hard-coded heuristics unless arbitrage is very high." Hard to test
- Computer use agents - Very early paradigm, requires wrangling. "You're getting models to do something they were definitely not even meant to do a year ago"

The takeaway: Start with Tier 1 patterns. Only graduate to Tier 2 when you've mastered the basics and have a genuine need.

Tips and Tricks

Some MCPs are Replaceable and Will Free Up Your Context Window

Here's how.

For MCPs such as version control (GitHub), databases (Supabase), deployment (Vercel, Railway) etc. - most of these platforms already have robust CLIs that the MCP is essentially just wrapping. The MCP is a nice wrapper but it comes at a cost.

To have the CLI function more like an MCP without actually using the MCP (and the decreased context window that comes with it), consider bundling the functionality into skills and commands. Strip out the tools the MCP exposes that make things easy and turn those into commands.

Example: instead of having the GitHub MCP loaded at all times, create a `/gh-pr` command that wraps `gh pr create` with your preferred options. Instead of the Supabase MCP eating context, create skills that use the Supabase CLI directly. The functionality is the same, the convenience is similar, but your context window is freed up for actual work.

This ties in with some of the other questions I've been getting. Over the past few days since I posted the original article, Boris and the Claude Code team has made a lot of progress in memory management and optimization, primarily with lazy loading of MCPs so that they don't eat your window from the start anymore. Previously I would've recommended converting MCPs into skills where you can, offloading the functionality to enact an MCP in one of two ways: by enabling it at that time (less ideal since you need to leave and resume session) or by having skills that use the CLI analogues to the MCP (if they exist) and having the skill be the wrapper around it - essentially having it act as a pseudo-MCP.

With lazy loading, the context window issue is mostly solved. But token usage and cost is not solved in the same way. The CLI + skills approach is still a token optimization method that may have results on par or near the effectiveness of using an MCP. Furthermore you can run MCP operations via CLI instead of in-context which reduces token usage significantly, especially useful for heavy MCP operations like database queries or deployments.

VIDEO?

As you suggested I'm thinking this paired with some of the other questions warrants a video to go alongside this article which covers these things.

Cover an END-TO-END PROJECT utilizing tactics from both articles:

- Full project setup with configs from the shorthand guide
- Advanced techniques from this longform guide in action
- Real-time token optimization
- Verification loops in practice
- Memory management across sessions
- The two-instance kickoff pattern
- Parallel workflows with git worktrees
- Screenshots and recordings of actual workflow

I'll see what I can do.

References

- [Anthropic: Demystifying evals for AI agents](
https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

) (Jan 2026)
- Anthropic: "Claude Code Best Practices" (Apr 2025)

- Fireworks AI: "Eval Driven Development with Claude Code" (Aug 2025)

- [YK: 32 Claude Code Tips](
https://agenticcoding.substack.com/p/32-claude-code-tips-from-basics-to
) (Dec 2025)
- Addy Osmani: "My LLM coding workflow going into 2026"

-
@PerceptualPeak
: Sub-Agent Context Negotiation
-
@menhguin
: Agent Abstractions Tierlist
-
@omarsar0
: Compound Effects Philosophy
- [RLanceMartin: Session Reflection Pattern](
https://rlancemartin.github.io/2025/12/01/claude_diary/
)
-
@alexhillman
: Self-Improving Memory System
Want to publish your own Article?

Upgrade to Premium
•
cogsec

@affaanmustafa

democratizing gambling
@pmx_trade
= (Ω , $\mathcal{F}$, $\mathbb{P}$_α); p ≈ $\mathbb{P}$(e=1); y ∈ {0, 1}; ∂p/∂α > 0